IT628 (Winter 2024-25), Lab 5: Threads

To get checked off on this lab, you have to solve Tasks 1 and 2, and show your solutions to a TA.

In pthreads, new threads are created by calling the function `pthread_create`. The actual definition of `pthread_create` is quite verbose:

```
int pthread_create(pthread_t *thread, const pthread_attr_t
*attr, void *(*start_routine)(void *), void *arg);
```

Let's start at the beginning. `pthread_create` returns an `int`, which is used to determine if a thread was successfully created (0 for yes, non-zero for no).

- The first argument is a pointer to a `pthread_t` structure. This structure holds all kinds of useful information about a thread (mostly used by the threading internals). So, for each thread you create, you'll need to allocate one of these structures to store all of its information.
- The second argument is a pointer to a `pthread_attr_t struct`. This structure contains the attributes of the thread. This has even more information about the thread. The only really interesting thing, for us at least, is that the `pthread_attr_t` struct tells the system how much memory to allocate for the thread's stack. By default, this is usually 512K. This is actually quite large, and if you run with lots of threads you can run out of memory just from allocating thread stacks.
- The third argument to `pthread_create` is even more mysterious. Lets look at it again: `void *(*start_routine)(void *)`. This is actually the function that the thread should start executing. This argument is an instance of what C programmers call a function pointer. The name of the argument is actually `start_routine`, and it is a function that returns `void *` and takes a `void *` as an argument.
- The last argument to `pthread_create` is a `void *` which is the argument to be passed to `start_routine` when the thread gets going. So, why a `void *`? Because in C, `void *` basically means anything. Remember you can cast pointers to `void *`, but you can also cast `int`s, `char`s, etc. to a `void *`. Which means that it is really the only way to specify a generic argument in C.

Review 1: Race conditions

The program below (example-1.c) is written to achieve the following features:

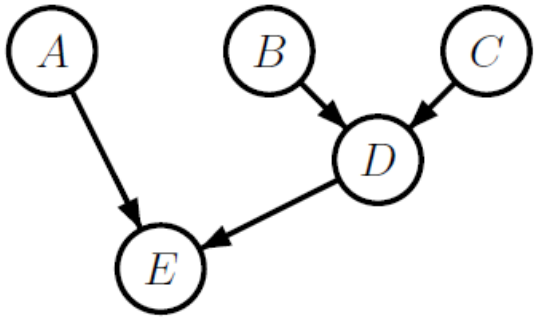
- Create 10 threads
- The thread number (NOT the thread id) is passed as an argument to the thread function
- Each thread should print its corresponding thread number inside its function
- Main thread should wait for all threads to complete. Program should terminate after this.

Compile and run the code. Check whether all the four features are successfully implemented. If not, try to reason why it is so?

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUMBER_OF_THREADS 10

void *print_hello_world(void * tnum) {
     printf("Hello World. Greetings from thread %d\n",
*(int*)tnum);
}

int main() {
   pthread_t threads[NUMBER_OF_THREADS];
   int status, i;

   for (i = 0; i < NUMBER_OF_THREADS; i++){
      status = pthread_create(&threads[i], NULL,
print_hello_world,
                              (void *) &i);
      if (status != 0) {
         printf("Oops, pthread_create returned error code %d\n",
                 status);
         exit(-1);
      }
   }

   for (i = 0; i < NUMBER_OF_THREADS; i++) {
      pthread_join(threads[i], NULL);
   }
}
```

Review 2: Thread join

Suppose that we have five functions that together solve some problem. Suppose these functions, labeled A through E, depend on each other according to the following graph.



Each edge of the graph denotes a dependency between two of these functions. For example, the edge from node B to node D means that functionB must be called, and must return, before functionD can be called.

What is wrong with this program (example-2.c) that uses Pthreads to execute the five functions concurrently in a way that adheres to the above dependency graph? How would you improve this program (but still use five worker threads and only the Pthreads functions pthread create() and pthread join())?

```
void *thread1(void *vargp){ funcA(); }
void *thread2(void *vargp){ funcB(); }
void *thread3(void *vargp){ funcC(); }
void *thread4(void *vargp){ funcD(); }
void *thread5(void *vargp){ funcE(); }


int main()
{
   pthread_t tid1, tid2, tid3, tid4, tid5;
   pthread_create(&tid2, NULL, thread2, NULL);
   pthread_create(&tid3, NULL, thread3, NULL);
   pthread_join(tid2, NULL);
   pthread_join(tid3, NULL);
   pthread_create(&tid1, NULL, thread1, NULL);
   pthread_create(&tid4, NULL, thread4, NULL);
   pthread_join(tid1, NULL);
   pthread_join(tid4, NULL);
   pthread_create(&tid5, NULL, thread5, NULL);
   pthread_join(tid5, NULL);
}
```

Task 1 (checkoff – 1 pt)

Write a program (example-3.c) that will take a list of filenames on the command line. If the list is empty, print an error message and exit. The goal is to compute how many lines are in each file using a thread for each file.

Create a thread for each argument given on the command line. Each thread should open its filename. If the open succeeds, the thread should read the file and count the number of newlines. When EOF is reached, the thread should print its filename with the number of newlines in the file, and terminate the message with a newline. The thread can then exit.

Task 2 (checkoff – 1 pt)

Write a Pthreads program (example-4.c) to execute the above five functions in a way that is maximally concurrent, adheres to the above dependency graph, and uses only three threads in addition to the main() thread. Your solution should still use only pthread join() for synchronization.